Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

TD7 Sécurité
Injection de code

Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les attaquants et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n°2023-22 du 24 janvier 2023.*

Mise en place avec Docker

Pour ce TD, l'environnement (serveur Apache + PHP + MySQL) est fourni sous forme de conteneurs Docker. Aucune machine virtuelle n'est nécessaire.

Prérequis : installer Docker

Sous Ubuntu/Debian :

```
# Installation de Docker
sudo apt update
sudo apt install -y docker.io docker-compose-v2
# Ajouter votre utilisateur au groupe docker
sudo usermod -aG docker $USER
# Deconnectez-vous puis reconnectez-vous pour
# que le changement de groupe prenne effet
```

Sous macOS/Windows : installer Docker Desktop.

Lancement de l'environnement

```
# Se placer dans le repertoire du TD
cd travaux-diriges/04-injection-code/td07/

# Lancer les conteneurs (premiere fois : ~2 min)
docker compose up -d

# Verifier que tout fonctionne
docker compose ps
```

Au premier lancement, Docker :

- construit un conteneur Apache+PHP avec l'extension `mysqli`,
- lance un conteneur MySQL 8.0 avec la base `l2info`,
- charge automatiquement les tables et données depuis `bd.sql`.

Accès aux pages web

Les fichiers du TD sont accessibles dans le navigateur à l'adresse :

`http://localhost:8080/td7/`

Par exemple : `http://localhost:8080/td7/page-5-commentaires.php`

Arrêt et nettoyage

```
# Arreter les conteneurs (les donnees sont conservees)
docker compose stop

# Relancer
docker compose start

# Arreter et supprimer tout (conteneurs + donnees)
docker compose down -v
```

Attention

Dans les fichiers PHP, le serveur MySQL est accessible via le nom d'hôte `"db"` (le nom du service Docker), et non `"localhost"`. C'est le fonctionnement normal de Docker : chaque service est un conteneur séparé, identifié par son nom.

Exercice 1 (Manipulation de formulaire HTML)**.** On considère une application web d'achat en ligne dont le formulaire HTML est le suivant (fichier `achat-iphone5.html`) :

```
<html>
<body>

<form method="post" action="enregistre-achat.php">
Produit: iphone5 <br/>
Prix : 500 euros <br/>
Quantit&eacute <input type="text" name ="quantite">
  (Quantite max est 90) <br/>
<input type="hidden" name="prix" value="500" >
<input type="submit" value="buy">
</form>

</body>
</html>
```

Le script PHP `enregistre-achat.php` enregistre l'achat en base de données en utilisant les valeurs envoyées par le formulaire.

1. Analysez le formulaire HTML. Identifiez la vulnérabilité liée à la manière dont le prix est transmis au serveur.
2. Proposez une méthode pour acheter un iPhone au prix de 5 € au lieu de 500 €.
3. Proposez une correction côté serveur pour empêcher cette attaque.

Point clé à retenir

Les champs `hidden` et toute donnée côté client peuvent être modifiés par l'utilisateur. La validation doit toujours se faire côté serveur.

Rappel de cours

Il existe trois types de XSS :

Exercice 2 (Cross-Site Scripting (XSS))**.**

- **XSS réfléchi (reflected)** : le payload est inclus dans la requête (URL, paramètre GET/POST) et renvoyé immédiatement dans la réponse sans être stocké.
- **XSS stocké (stored)** : le payload est enregistré sur le serveur (base de données, fichier) et affiché à tous les visiteurs ultérieurs de la page.
- **XSS basé sur le DOM (DOM-based)** : le payload est interprété directement par le JavaScript côté client, sans transiter par le serveur.

a) XSS stocké – alerte JavaScript

On considère la page `page-5-commentaires.php` dont le code est le suivant :

```
<html>
<H1> Computer Security  </H1>
<IMG src="img_security.jpeg"> Computer security...
<BR/><hr/>
<h3> Visitors comments :</h3>
```

```php
<?php
$conn = new mysqli("db", "l2info", "l2info", "l2info");
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

if( isset($_POST['login']) && isset($_POST['commentaire']) )
{
    $sql = 'INSERT INTO Commentaires(login,commentaire)
      VALUES("' . $_POST['login']. '","'
      . $_POST['commentaire']."');";
    $result0 = $conn->query($sql);
}

$sql = 'SELECT id,login, commentaire FROM Commentaires
  ORDER BY id DESC LIMIT 4 ;';
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    echo "<ul>";
    while($row = $result->fetch_assoc()) {
        echo "<li>";
        echo "[" . $row["login"]. "]: "
          . $row["commentaire"]."<br>";
        echo "</li>";
    }
    echo "</ul>";
} else {
    echo "Aucun commentaire.";
}
$conn->close();
?>
<BR/><hr/><BR/>
<form name="postcomment" method="POST"
  action="page-5-commentaires.php">
Pseudo : <input name="login"> <BR/>
Commentaire : <input type="text" name="commentaire"><BR/>
<input type="submit">
</form>
</html>
```

1. Expliquez ce que fait cette page.
2. Proposez une injection dans le champ commentaire pour qu'un message d'alerte JavaScript s'affiche au chargement de la page.

b) XSS stocké – défacement

En utilisant la même page, injectez du HTML frauduleux pour faire apparaître une fausse section Références avec un lien vers un site fictif.

c) XSS réfléchi via GET

On considère le code PHP suivant :

```
<html>
<body>
<?php
if( isset($_GET['UserInput'])){
  $out ='vous avez saisi: "'
    . $_GET['UserInput'] . '".';
}
else {
  $out = '<form method="GET">
    saisissez quelque chose ici :';
  $out .= '<input name="UserInput" size="50">';
  $out .= '<input type="submit">';
  $out .= '</form>';
}
print $out;
?>
</body>
</html>
```

Soumettez une entrée qui fait apparaître une alerte JavaScript dans le résultat.

d) XSS réfléchi via POST

On considère maintenant le code suivant :

```
<html>
<body>
<?php
if( isset($_POST['UserInput'])){
  $out ='vous avez saisi: "'
    . $_POST['UserInput'] . '".';
}
else {
  $out='Aucune donnee transmise au script';
}
print $out;
?>
</body>
</html>
```

Construisez une page HTML externe contenant un formulaire POST caché qui soumet automatiquement un payload XSS à cette page.

e) XSS basé sur le DOM

On considère la page HTML suivante :

```
<html>
<body>
<div id="welcome"></div>
<script>
  var name = document.location.hash.substring(1);
  document.getElementById("welcome").innerHTML
    = "Bienvenue " + name;
</script>
</body>
</html>
```

1. Construisez une URL qui exploite cette vulnérabilité pour exécuter du JavaScript.
2. Expliquez pourquoi ce type de XSS est plus difficile à détecter côté serveur.

Attention

Le XSS stocké est le plus dangereux car il affecte tous les visiteurs de la page, sans nécessiter de lien piégé.

Rappel de cours

Différence entre XSS et injection SQL :

Exercice 3 (Injection SQL).

- Le **XSS** injecte du code malveillant dans le *navigateur de la victime* (côté client). L'attaque vise les utilisateurs du site.
- L'**injection SQL** injecte du code malveillant dans la *base de données du serveur* (côté serveur). L'attaque vise directement les données du site.

a) Analyse du code vulnérable

On considère la page `page-transaction-sql.php` :

```
<html>
<?php
$servername = "db";
$username = "l2info";
$password = "l2info";
$dbname = "l2info";

$conn = new mysqli($servername, $username,
  $password, $dbname);
if ($conn->connect_error) {
  die("Connection failed: "
    . $conn->connect_error);
}
echo 'Nom : "' . $_POST['UserInput'].'"<BR/>';
$sql = 'SELECT montant FROM Transaction WHERE nom="'
  . $_POST['UserInput']  . '";';
echo "Requete SQL=" . $sql . "<BR/>";
$result = $conn->query($sql);
echo "Resultat:<BR/>";
if ($result->num_rows > 0) {
  while($row = $result->fetch_assoc()) {
    echo "Nom: ". $_POST['UserInput']
      ." Montant :" . $row["montant"]."<br/>";
  }
} else {
  echo "0 results";
}
$conn->close();
?>
</html>
```

Et le formulaire `injection-sql.html` :

```html
<html>
<body>
<form name="exploit-sql" method="POST"
  action="http://localhost:8080/td7/page-transaction-sql.php">
<input name="UserInput">
<input type="submit">
</form>
</body>
</html>
```

1. Expliquez comment la requête SQL est construite. Identifiez la vulnérabilité.

b) Afficher toute la table

Proposez une injection pour afficher l'intégralité de la table `Transaction`. Expliquez pas-à-pas la requête résultante.

c) Tentative de DROP TABLE

Tentez de supprimer la table `Test` par injection SQL. Expliquez pourquoi cela peut échouer avec MySQL.

d) UNION-based

Extraire les données de la table `Test` (colonne `val`) via une injection `UNION SELECT`.

Indice : commencez par déterminer le nombre de colonnes du `SELECT` original en utilisant `ORDER BY 1`, `ORDER BY 2`, etc. jusqu'à obtenir une erreur. Puis construisez le `UNION SELECT` avec le bon nombre de colonnes.

e) Réflexion sur le blind SQL

On suppose maintenant que la page n'affiche plus les résultats de la requête mais seulement Transaction trouvée ou Aucune transaction .

1. Comment pourrait-on déterminer si la version de MySQL est 5.x ? (concept *boolean-based blind SQL injection*)
2. Quel serait le principe pour extraire des données caractère par caractère ?

Point clé à retenir

L'injection SQL fonctionne car la requête mélange code SQL et données utilisateur dans une même chaîne. La solution est de les séparer avec des **requêtes préparées** (*prepared statements*).

Exercice 4 (Contre-mesures)**. a) Corriger le XSS**

Modifiez le code de `page-5-commentaires.php` pour empêcher l'injection XSS. Quelle fonction PHP permet d'échapper les caractères spéciaux HTML ? Donnez le code corrigé de la ligne d'affichage des commentaires.

b) Corriger l'injection SQL

Réécrivez `page-transaction-sql.php` en utilisant des requêtes préparées PDO au lieu de la concaténation avec `mysqli`. Donnez le code complet corrigé.

c) Défenses complémentaires

1. Qu'est-ce que le header HTTP `Content-Security-Policy` et comment protège-t-il contre le XSS ?
2. Qu'est-ce qu'un cookie `HttpOnly` et pourquoi est-ce utile ?
3. Pourquoi le principe du moindre privilège est-il important pour la base de données ?

Point clé à retenir

La défense en profondeur combine plusieurs mécanismes : échappement en sortie, requêtes préparées, CSP, cookies `HttpOnly`, validation des entrées.